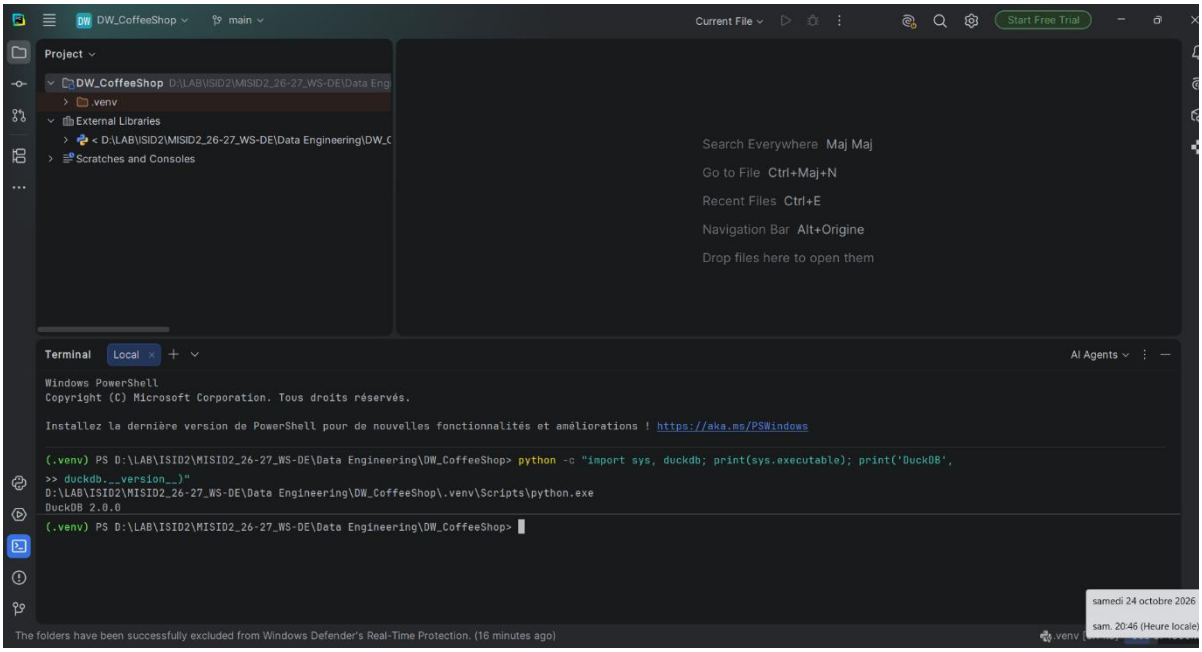
25 octobre 2026

Compte rendu de TP

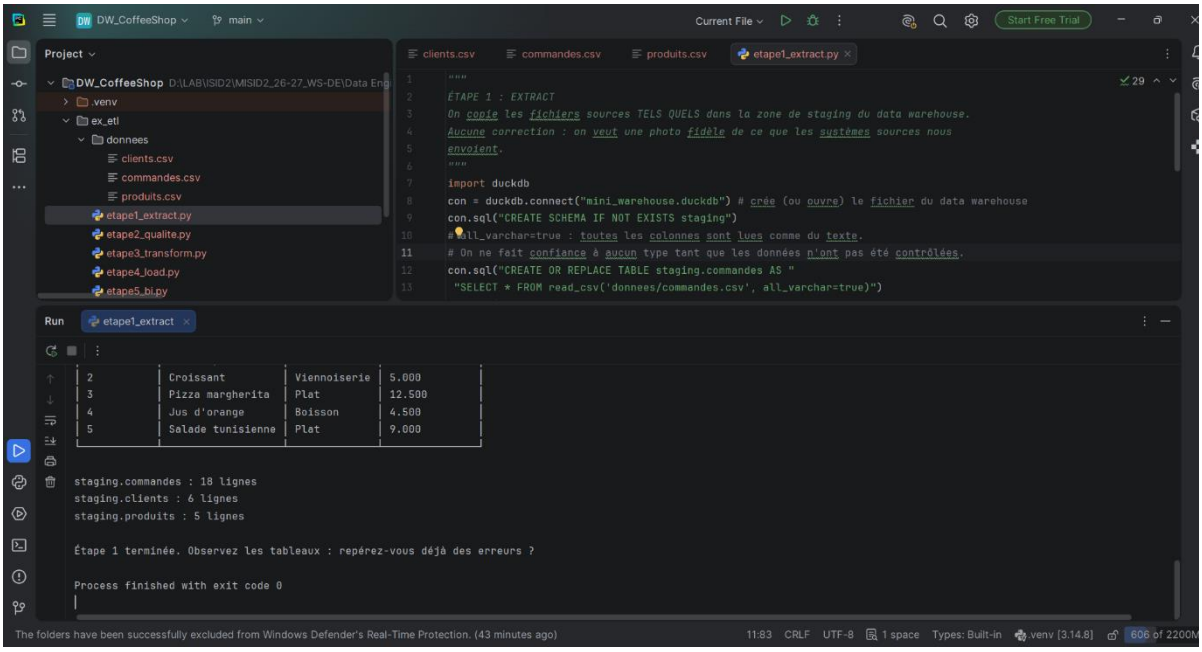
Construire un Data Warehouse avec un ETL

Zied.Brahmi
MISID2

Étape 0 : installer Python et configurer l'interpréteur du projet



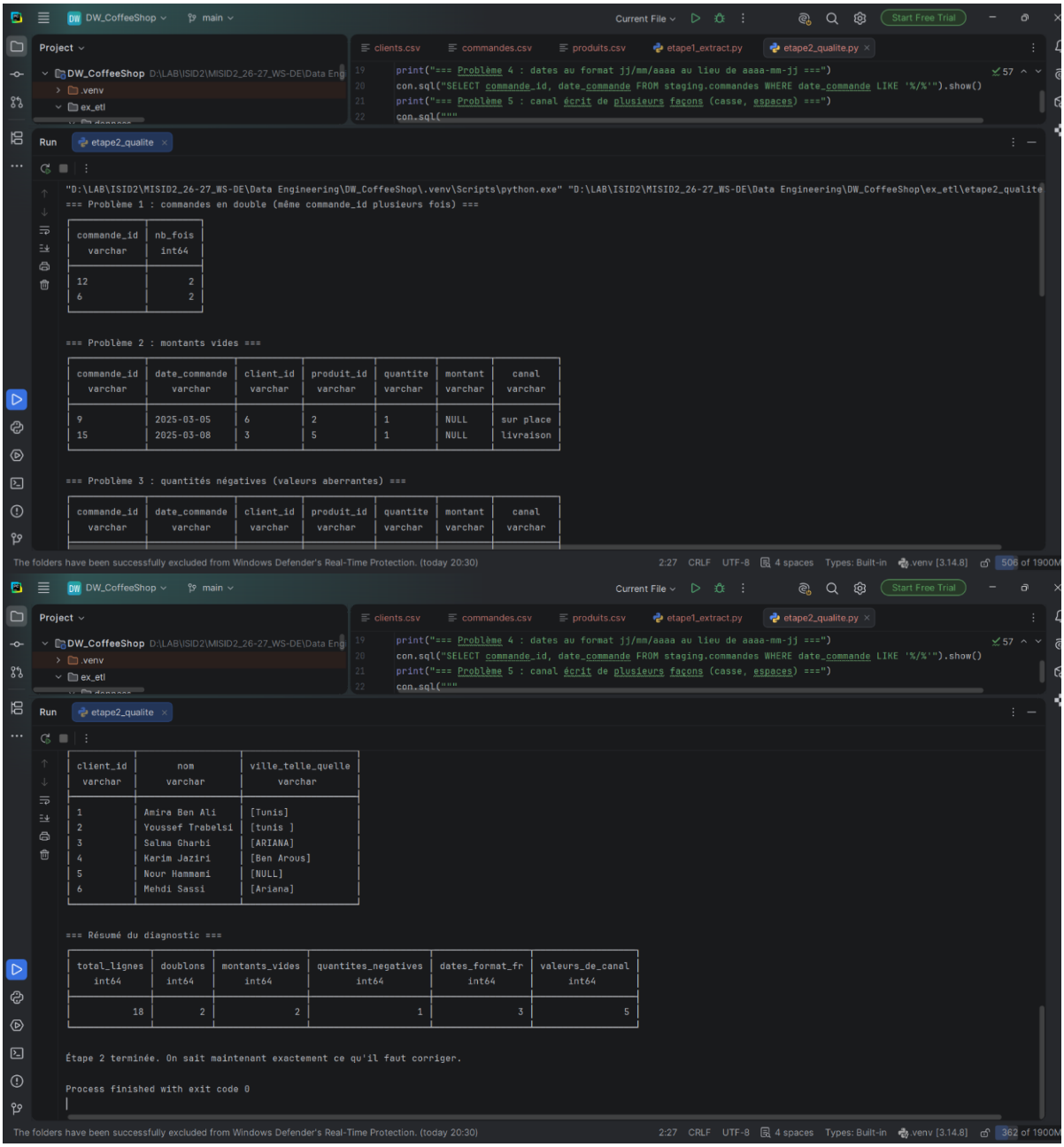
Étape 1 : Extract, charger les sources dans la zone de staging



Questions :

1. On charge n toutes les colonnes en texte (*all_varchar=true*) parce que nous n'avons pas confiance que les types de données sont respectés dans les données sources.
2. Dans *staging.commandes*, les lignes 6 et 12 apparaissent en doubles.
3. En lançant le script une deuxième fois, le nombre de lignes ne change pas parce nous avons utilisé la commande DDL SQL *CREATE OR REPLACE TABLE* qui va écraser l'ancienne table *staging.commandes*.

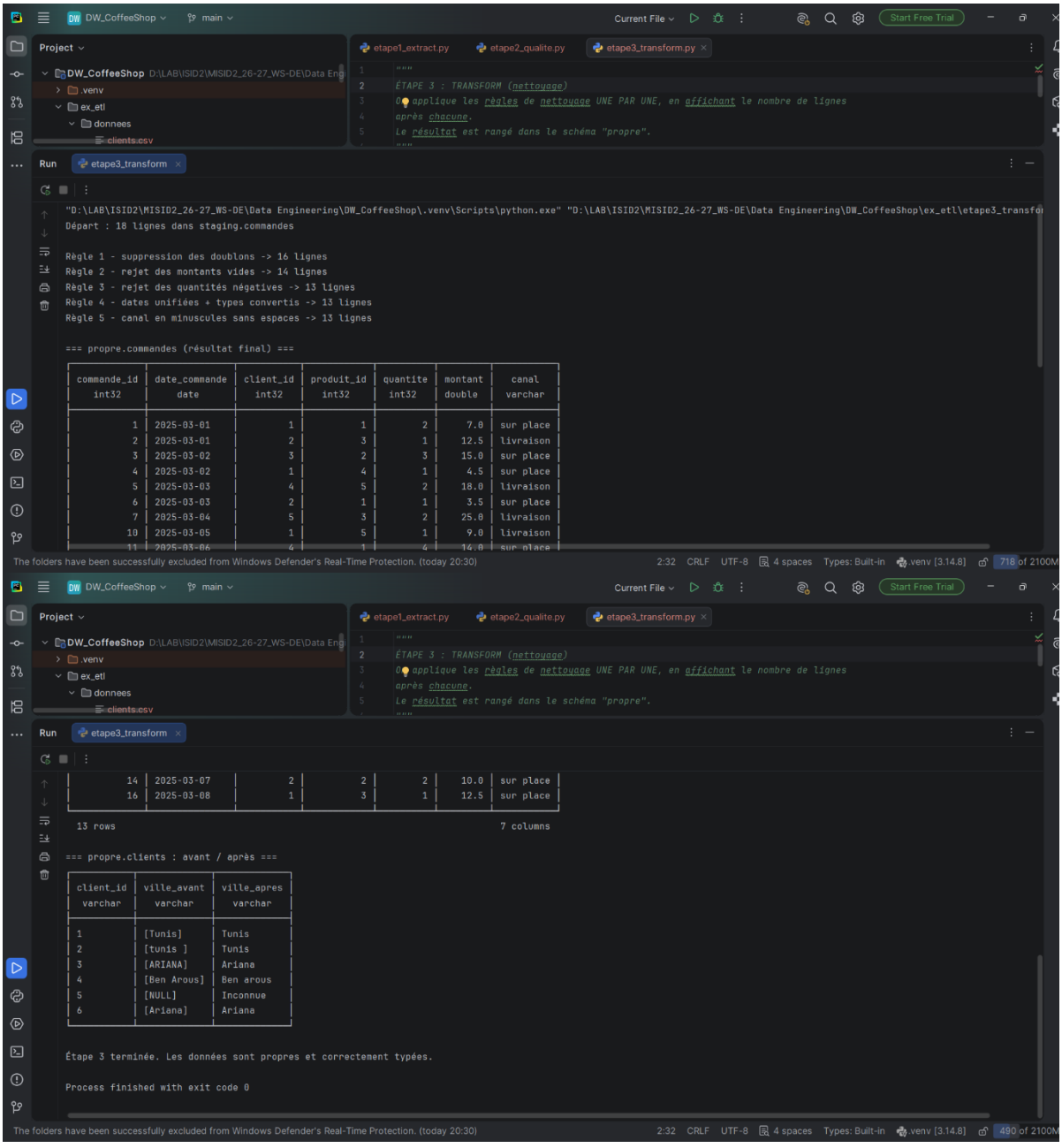
Étape 2 : contrôle qualité, diagnostiquer les erreurs



Questions :

1. Dans le contrôle visuel, j'ai surtout manqué les espaces.
2. Les espace dans *[sur place]* sont dangereux pour un rapport, parce qu'elle sera traité comme une autre valeur différente de *[sur place]* pour la dimension canal.
3. Il ne faut pas supprimer la cliente *Salma Gharbi* dont le numéro de téléphone n'est pas renseigné parce que cette information n'est pas utile pour le rapport ; elle n'est indispensable qu'en production pour le canal livraison.
4. Les doublons (6 & 12) et les valeurs aberrantes (8 : quantité négative) impacterons directement le CA, mais aussi les analyses suivant les axes Ville et Canal seront faussées par la façon dont les lignes de ces dimensions sont écrites.

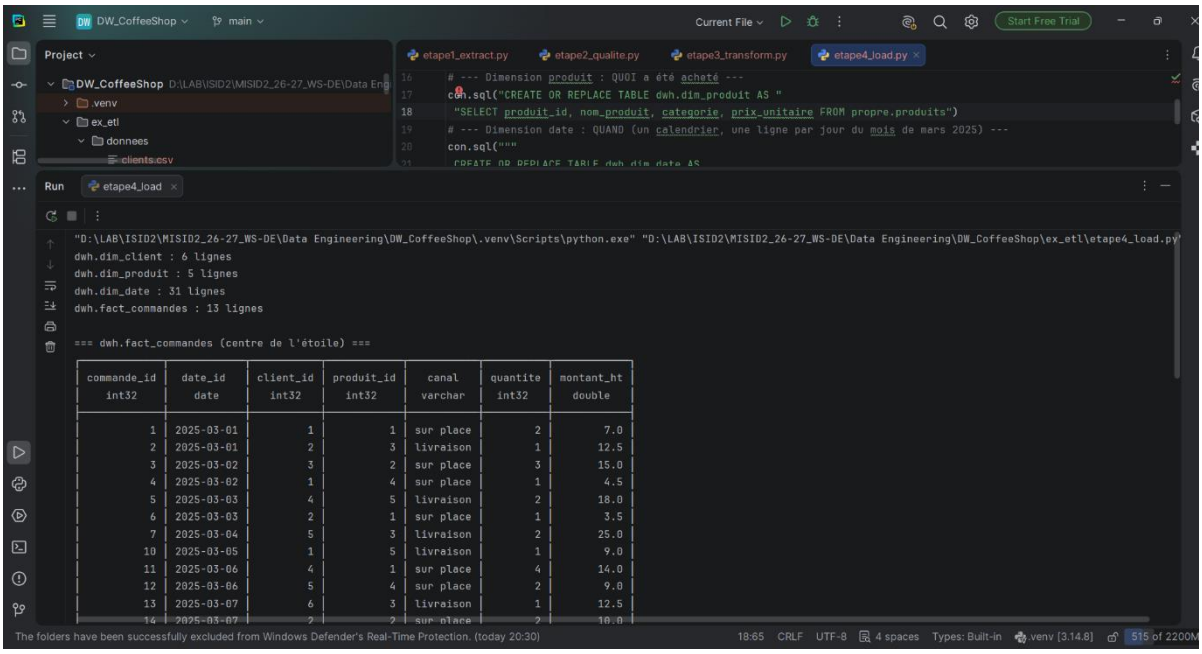
Étape 3 : Transform, nettoyer les données règle par règle

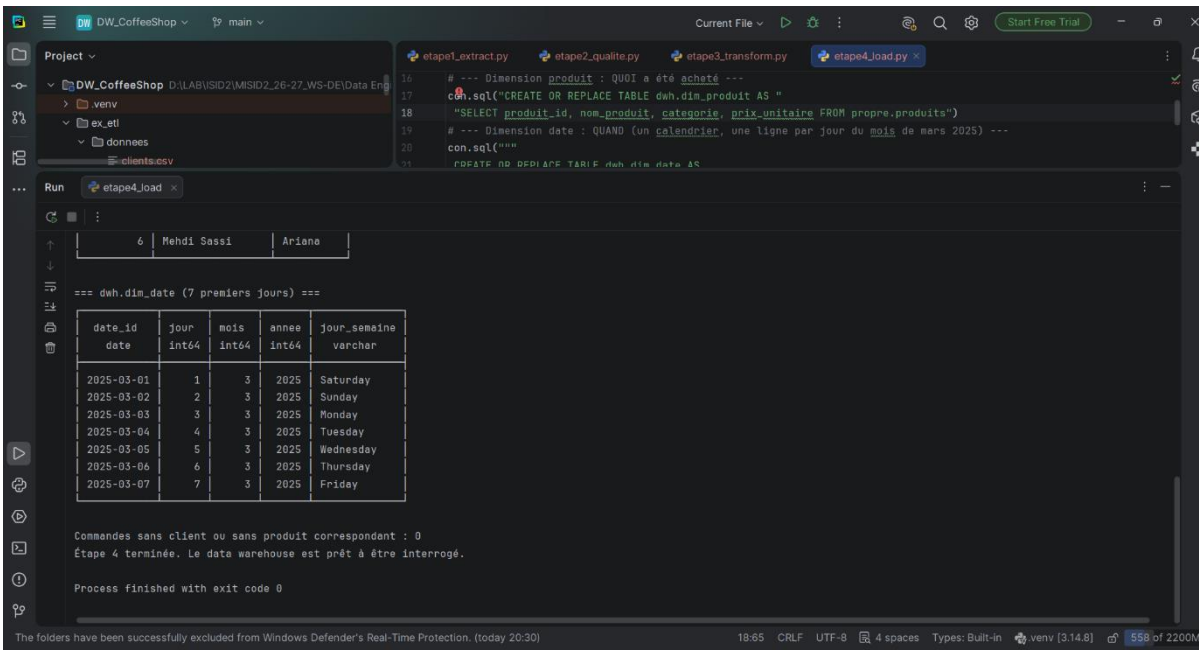


Questions :

1. Les règles 4 et 5 ne changent pas le nombre de lignes parce que ceux sont des opérations de mise en forme seulement : casting de types pour 4 et mise en minuscule sans espaces pour 5.
2. Si on essaie de faire le casting en INT d'une quantité :
    - Négative : pas de problèmes pour duckdb mais ceci faussera le reporting.
    - Non numérique : duckdb retourne une erreur
3. On a choisi de mettre *Inconnue* pour la colonne ville plutôt que supprimer les clients sans ville, pour ne pas perdre leurs transactions lors d'une analyse par ville.
4. Nous avons pu ajouter une règle pour contrôler le calcul du montant total à partir de prix_unitaire(produits) (qui est à contrôler aussi) et quantite(commandes).

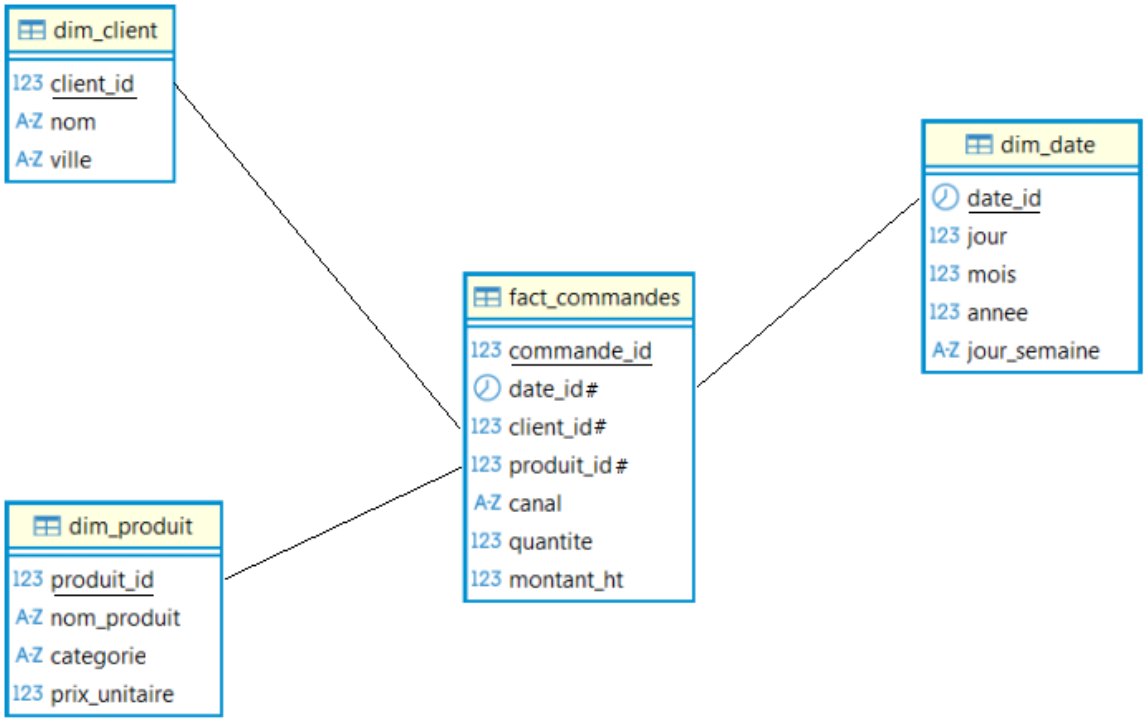
Étape 4 : Load, construire le schéma en étoile





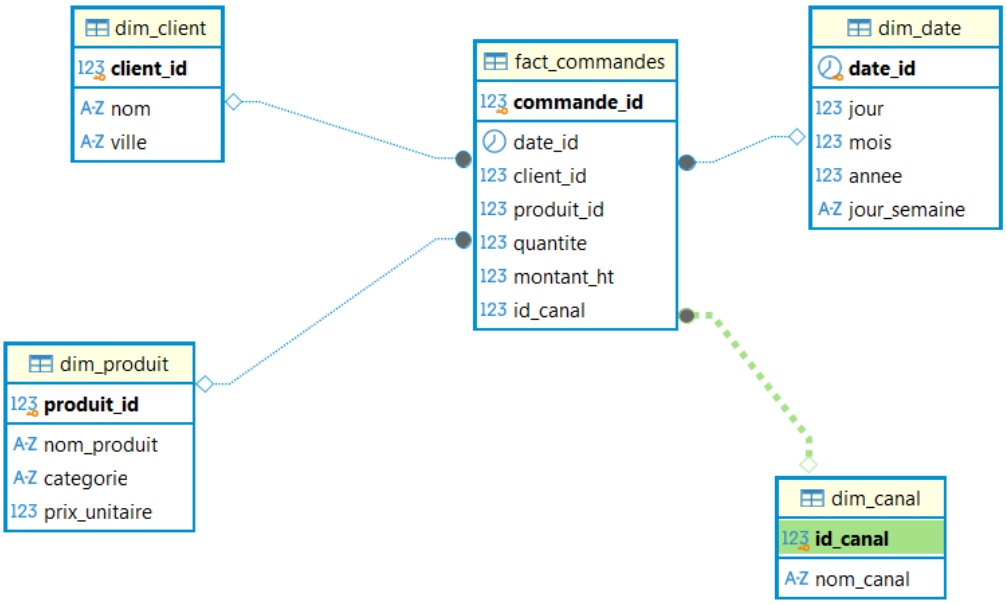
Questions :

1. La granularité de *fact_commandes* est une ligne par commande. Pour passer à une granularité par jour, il faudra remplacer la colonne *commande_id* par *nbr_commande* (nombre total de commandes pour un jour j) et grouper le montant, la quantité et le nombre de commandes par jour lors du chargement de la table des faits.
2. La table *dim_date* contient 31 lignes alors qu'il n'y a des commandes que sur 8 jours parce c'est une dimension temporelle indépendante de la table des faits. Elle serait utile pour faire une analyse qui visualise les jours sans activité.
3. Schéma du DataWarehouse :

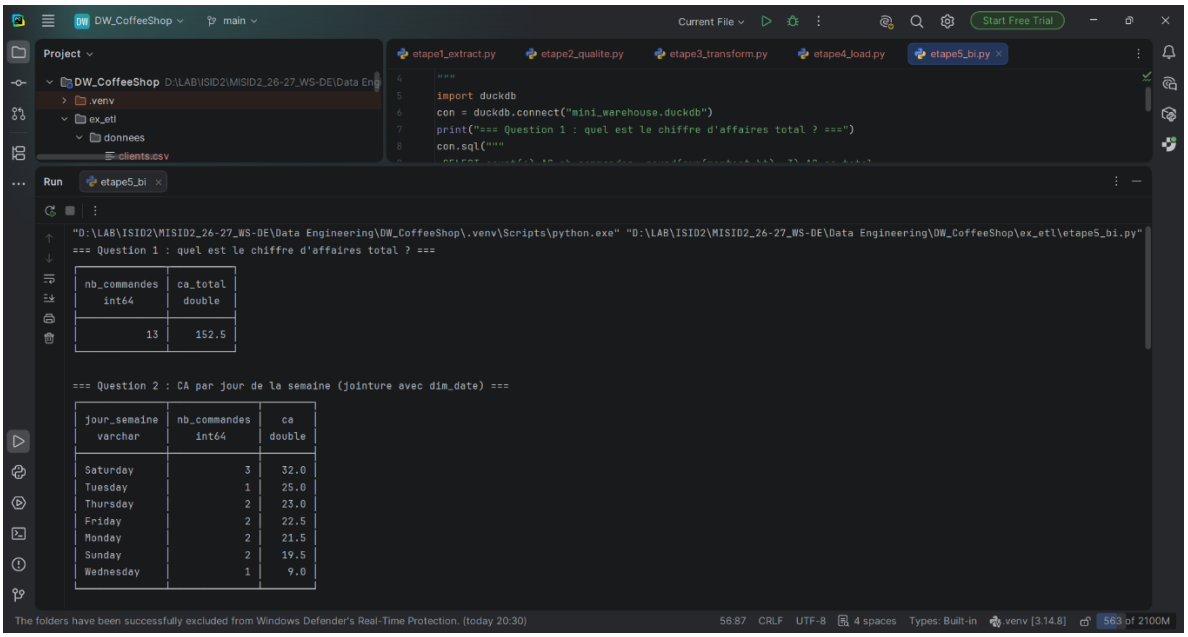


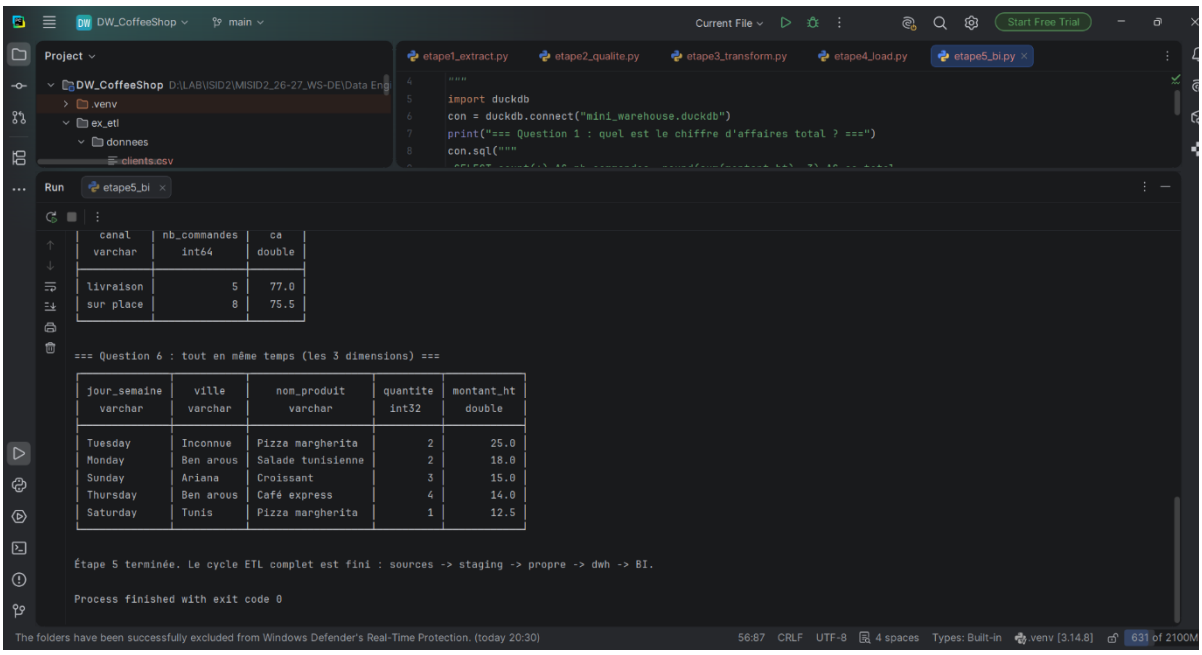
4. La colonne canal reste dans la table de faits plutôt que dans une dimension parce qu'elle possède seulement deux modalités ('livraison' et 'sur place') et elle est peu évolutive ce qui rend l'architecture du DWH plus simple et les requêtes plus performantes.

   Ceci est une proposition de la dimension canal :



Étape 5 : analyse BI, répondre aux questions du gérant

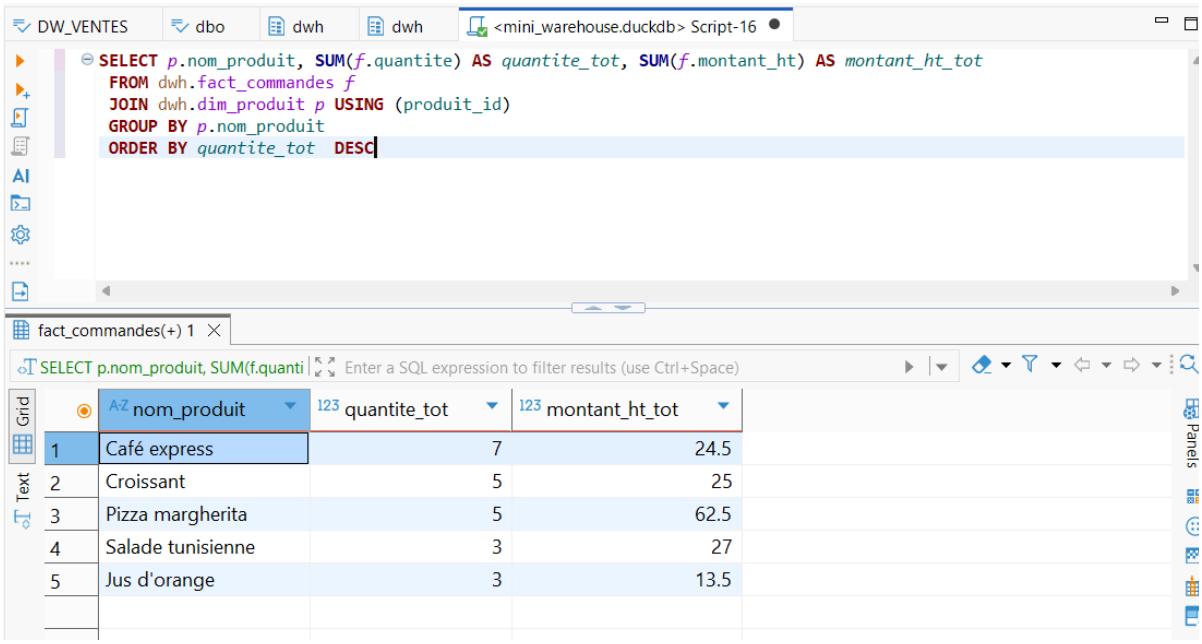




Questions :

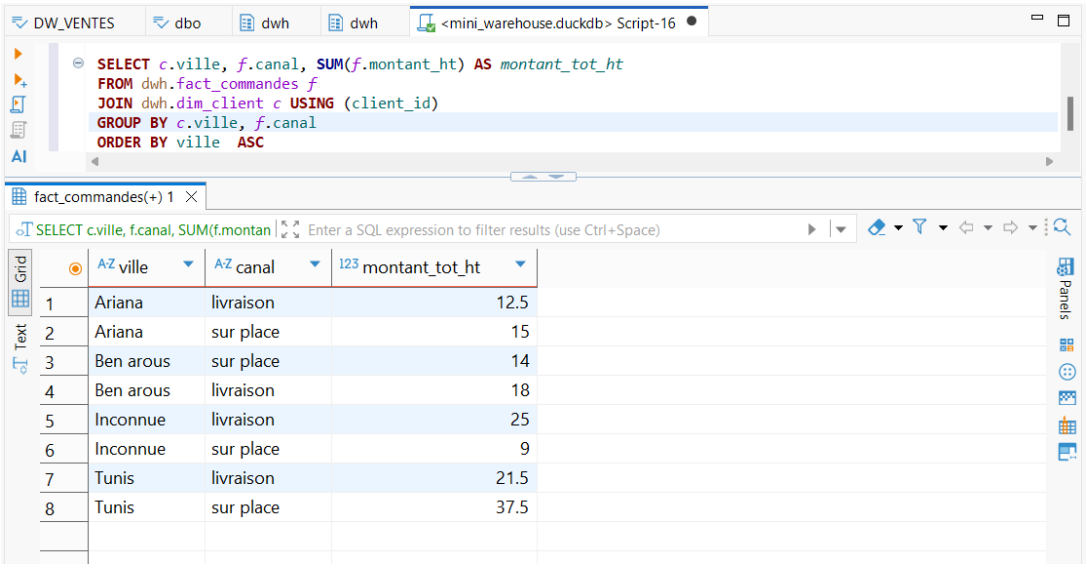
1. CA par produit (nom du produit), du plus vendu au moins vendu :

```
SELECT p.nom_produit, SUM(f.quantite) AS quantite_tot, SUM(f.montant_ht) AS
montant_ht_tot
 FROM dwh.fact_commandes f
 JOIN dwh.dim_produit p USING (produit_id)
 GROUP BY p.nom_produit
 ORDER BY quantite_tot  DESC
```



2. CA par ville et par canal :

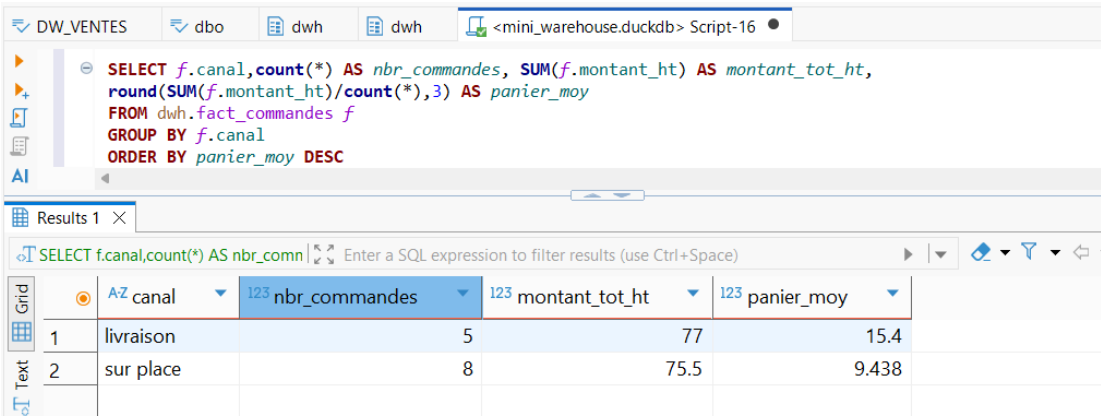
```
SELECT c.ville, f.canal, SUM(f.montant_ht) AS montant_tot_ht
FROM dwh.fact_commandes f
JOIN dwh.dim_client c USING (client_id)
GROUP BY c.ville, f.canal
ORDER BY ville  ASC
```



	ville	canal	montant_tot_ht
1	Ariana	livraison	12.5
2	Ariana	sur place	15
3	Ben arous	sur place	14
4	Ben arous	livraison	18
5	Inconnue	livraison	25
6	Inconnue	sur place	9
7	Tunis	livraison	21.5
8	Tunis	sur place	37.5

3. La ville Inconnue (34) est au second rang après Tunis (59), donc on recommande au gérant de mettre plus d'effort pour renseigner la ville du client pour donner plus de valeur aux analyses par cet axe.
4. Panier moyen :

```
SELECT f.canal,count(*) AS nbr_commandes, SUM(f.montant_ht) AS
montant_tot_ht,
round(SUM(f.montant_ht)/count(*),3) AS panier_moy
FROM dwh.fact_commandes f
GROUP BY f.canal
ORDER BY panier_moy DESC
```

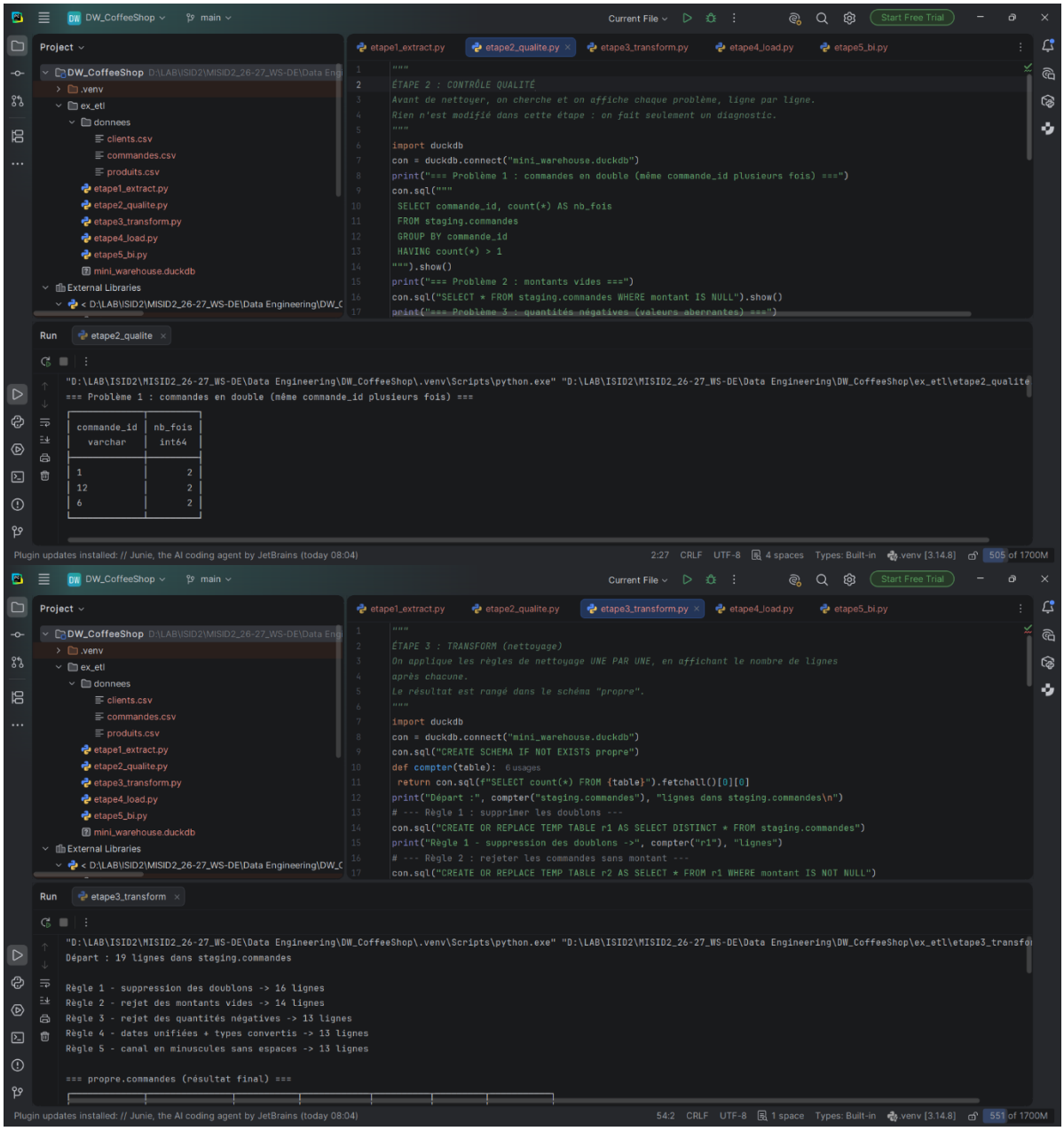


	canal	nbr_commandes	montant_tot_ht	panier_moy
1	livraison	5	77	15.4
2	sur place	8	75.5	9.438

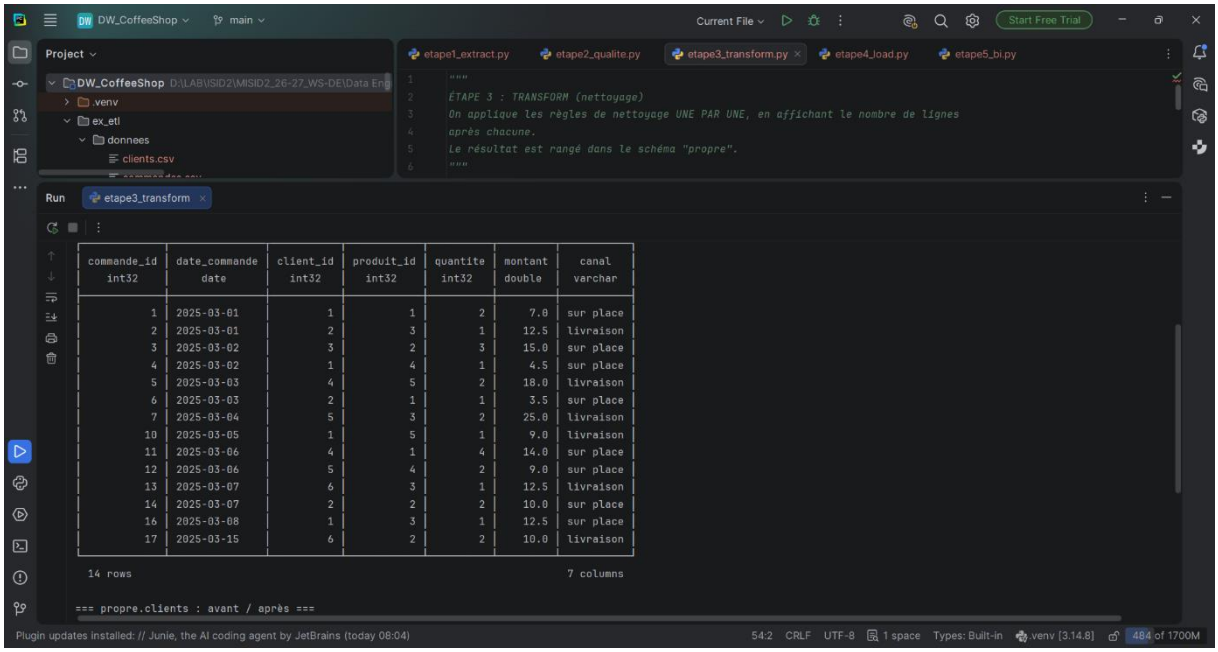
On constate que le panier moyen de la livraison est plus élevé que celui de sur place.

Exercices de manipulation

1. L'étape 2 détecte le nouveau doublon mais rien ne change dans l'étape 3 sauf le nombre de lignes de *staging.commandes* : 19

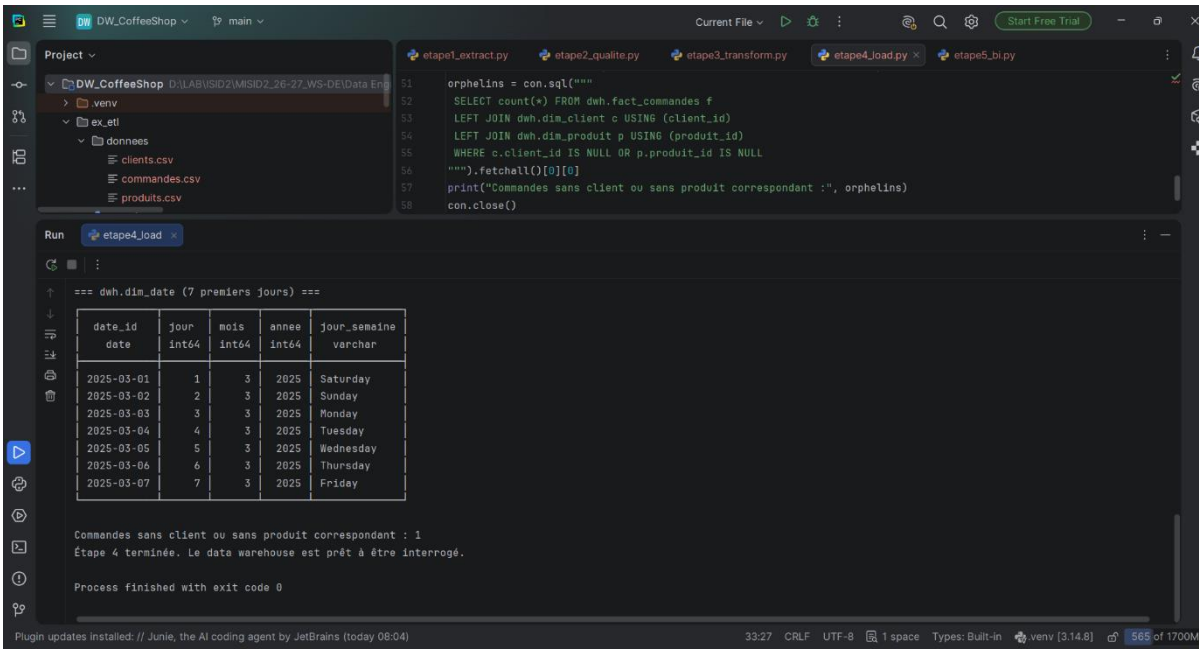


2. Nouvelle date française

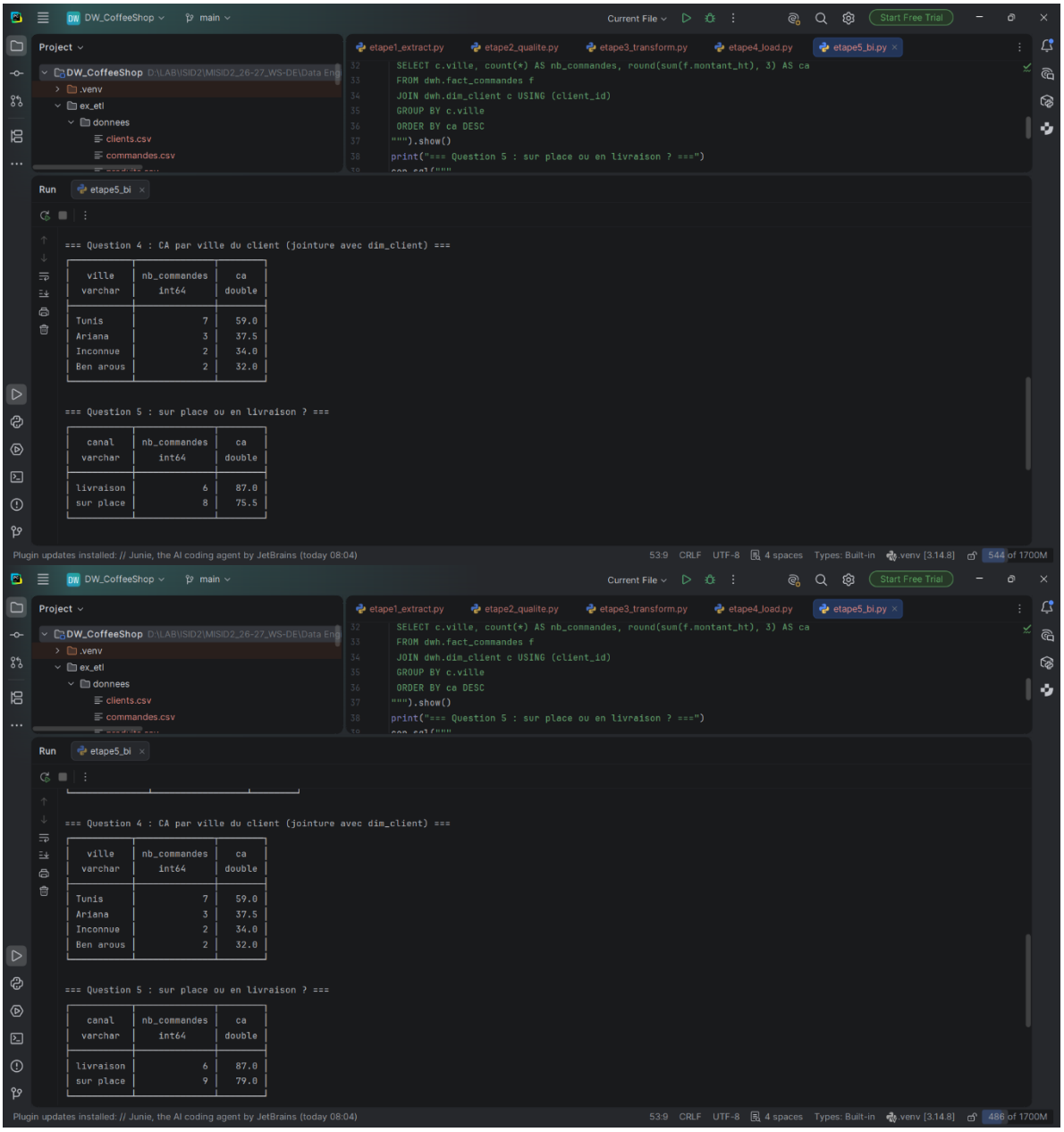


3. Client inconnu

L'étape 4 détecte une commande sans client ou produit correspondant.

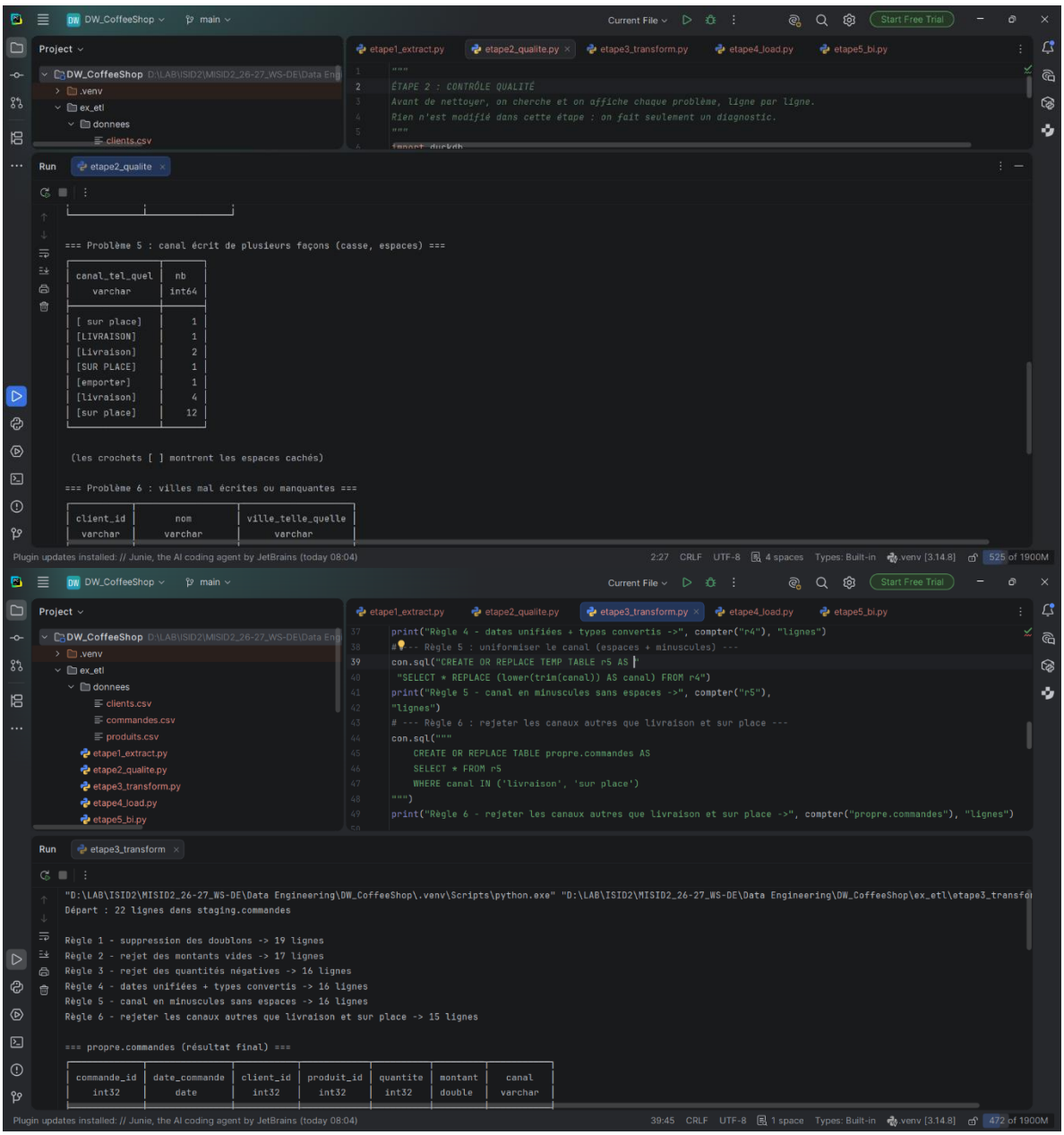


Le CA par ville de l'étape 5 ne change pas parce que la jointure sur *id_client* élimine la commande 18.



4. Nouvelle règle

```
# --- Règle 5 : uniformiser le canal (espaces + minuscules) ---
con.sql("CREATE OR REPLACE TEMP TABLE r5 AS "
 "SELECT * REPLACE (lower(trim(canal)) AS canal) FROM r4")
print("Règle 5 - canal en minuscules sans espaces ->", compter("r5"),
"lignes")
# --- Règle 6 : rejeter les canaux autres que livraison et sur place ---
con.sql("""
    CREATE OR REPLACE TABLE propre.commandes AS
    SELECT * FROM r5
    WHERE canal IN ('livraison', 'sur place')
""")
print("Règle 6 - rejeter les canaux autres que livraison et sur place ->",
compter("propre.commandes"), "lignes")
```


DW_CoffeeShop
main
Current File
Start Free Trial
Project
DW_CoffeeShop D:\LAB\ISID2\MISID2_26-27_WS-DE\Data Eng
.venv
ex_etl
donnees
clients.csv
etape1_extract.py
etape2_qualite.py
etape3_transform.py
etape4_load.py
etape5_bi.py
"""
ÉTAPE 2 : CONTRÔLE QUALITÉ
Avant de nettoyer, on cherche et on affiche chaque problème, ligne par ligne.
Rien n'est modifié dans cette étape : on fait seulement un diagnostic.
"""
Run
etape2_qualite
=== Problème 5 : canal écrit de plusieurs façons (casse, espaces) ===
canal_tel_quel | nb
varchar | int64
[sur place] | 1
[LIVRAISON] | 1
[Livraison] | 2
[SUR PLACE] | 1
[emporter] | 1
[livraison] | 4
[sur place] | 12
(les crochets [ ] montrent les espaces cachés)
=== Problème 6 : villes mal écrites ou manquantes ===
client_id | nom | ville_telle_quelle
varchar | varchar | varchar
Plugin updates installed: // Junie, the AI coding agent by JetBrains (today 08:04)
2:27 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.8] 525 of 1900M

DW_CoffeeShop
main
Current File
Start Free Trial
Project
DW_CoffeeShop D:\LAB\ISID2\MISID2_26-27_WS-DE\Data Eng
.venv
ex_etl
donnees
clients.csv
commandes.csv
produits.csv
etape1_extract.py
etape2_qualite.py
etape3_transform.py
etape4_load.py
etape5_bi.py
print("Règle 4 - dates unifiées + types convertis ->", compter("r4"), "lignes")
--- Règle 5 : uniformiser le canal (espaces + minuscules) ---
con.sql("CREATE OR REPLACE TEMP TABLE r5 AS "
"SELECT * REPLACE (lower(trim(canal)) AS canal) FROM r4")
print("Règle 5 - canal en minuscules sans espaces ->", compter("r5"),
"lignes")
--- Règle 6 : rejeter les canaux autres que livraison et sur place ---
con.sql("""
CREATE OR REPLACE TABLE propre.commandes AS
SELECT * FROM r5
WHERE canal IN ('livraison', 'sur place')
""")
print("Règle 6 - rejeter les canaux autres que livraison et sur place ->", compter("propre.commandes"), "lignes")
Run
etape3_transform
"D:\LAB\ISID2\MISID2_26-27_WS-DE\Data Engineering\DW_CoffeeShop\.venv\Scripts\python.exe" "D:\LAB\ISID2\MISID2_26-27_WS-DE\Data Engineering\DW_CoffeeShop\ex_etl\etape3_transfo
Départ : 22 lignes dans staging.commandes
Règle 1 - suppression des doublons -> 19 lignes
Règle 2 - rejet des montants vides -> 17 lignes
Règle 3 - rejet des quantités négatives -> 16 lignes
Règle 4 - dates unifiées + types convertis -> 16 lignes
Règle 5 - canal en minuscules sans espaces -> 16 lignes
Règle 6 - rejeter les canaux autres que livraison et sur place -> 15 lignes
=== propre.commandes (résultat final) ===
commande_id | date_commande | client_id | produit_id | quantite | montant | canal
int32 | date | int32 | int32 | int32 | double | varchar
Plugin updates installed: // Junie, the AI coding agent by JetBrains (today 08:04)
39:45 CRLF UTF-8 1 space Types: Built-in .venv [3.14.8] 472 of 1900M